

WAL-G Internals: A Deep Technical Report for PostgreSQL Specialists

TL;DR

- WAL-G is an Apache-2.0, Go-based physical backup/archival/restore engine for PostgreSQL (plus MySQL/MariaDB, MS SQL Server, MongoDB, Redis, Greenplum/Cloudberry, FoundationDB and etcd), successor to Daniel Farina's WAL-E, built around a pluggable pipeline of storage backends, compressors and crypters wired through a Cobra/Viper CLI.
- Its distinguishing PostgreSQL internals are: non-exclusive base backups via `pg_backup_start/stop`, concurrent tar composition (regular/rating/copy/database composers) coordinated by a `TarBallQueue`, page-level delta backups using an `IncrementalPageReader` diff map keyed on page LSNs, optional WAL-scanning delta maps (roaring bitmaps), reverse-delta unpack on fetch, and a `wal-verify` integrity scanner.
- The codebase is organized as `cmd/<db>` (per-engine Cobra command trees), `internal/` (handlers + `internal/databases/<engine>` logic + `internal/walparser`, `internal/compression`, `internal/crypto`, `internal/multistorage`) and `pkg/storages/*` (the S3/GCS/Azure/OSS/Swift/FS/SSH abstraction), producing one binary per engine via `make <db>_build` with cgo build tags for `brotli/libsodium/lzo`.

Key Findings

Language & build. WAL-G is ~87% Go (GitHub language breakdown: Go 87.4%, Shell 10.4%, plus Dockerfile/Gherkin/Makefile/Python). The master `go.mod` declares `module github.com/wal-g/wal-g` with a Go 1.25.x directive and `toolchain go1.25.9`. Binaries are built per engine (`make pg_build`, `mysql_build`, `sqlserver_build`, `redis_build`, `mongo_build`, `gp_build`, `cloudberry_build`) as `go build -mod vendor -tags "$(BUILD_TAGS)"`, with optional cgo features toggled by `USE_BROTLI`/`USE_LIBSODIUM`/`USE_LZO` environment variables that append `brotli`, `libsodium`, `lzo` build tags. Released binaries are named `wal-g-DBNAME-OSNAME`.

Architecture. The core abstraction is a streaming pipeline: `CompressAndEncrypt(source, compressor, crypter)` returns a pipe reader that is uploaded; on the way back `DecryptAndDecompressTar` reverses it. Storage, compression and encryption are all interface-based and independently pluggable.

Commands. For PostgreSQL: `backup-push`, `backup-fetch`, `backup-list`, `backup-mark`, `delete` (retain/before/everything/target/garbage), `wal-push`, `wal-fetch`, `wal-prefetch`, `wal-show`, `wal-verify`, `wal-receive`, `wal-restore`, `catchup-push`, `catchup-fetch`, `catchup-send`, `catchup-receive`, `catchup-list`, `copy`, `daemon`, `st` (storage tools), plus `pgbackrest` interop commands.

Details

1. What WAL-G is, purpose, and relationship to WAL-E

WAL-G is described in its own README as "an archival restoration tool for PostgreSQL, MySQL/MariaDB, and MS SQL Server (beta for MongoDB and Redis)," and its GitHub tagline is "Archival and Restoration for databases in the Cloud." It performs continuous WAL archiving plus periodic physical base backups to object storage, enabling point-in-time recovery (PITR). Beyond Postgres it supports MySQL/MariaDB (stream + xtrabackup), MS SQL Server, MongoDB (beta, including binary backups), Redis (beta, including sharded), Greenplum/Cloudberry (production), and work-in-progress FoundationDB and etcd. [GitHub](#)

It is the successor to **WAL-E**, the Python continuous-archiving tool originally authored by Daniel Farina. WAL-G was created in 2017 as a collaboration between Farina (by then a principal engineer on the Citus Cloud team and original WAL-E author) and Katie Li, a Citus Data software-engineering intern; it was introduced in the Citus Data blog post "Introducing WAL-G by Citus: Faster Disaster Recovery for Postgres" (2017-08-18). Development is now supported by Yandex Cloud. Key differences from WAL-E, per the Citus post and README: WAL-G is a Go rewrite that uses LZ4/LZMA/ZSTD/Brotli compression, multiple processors (parallelism), and **non-exclusive** base backups. It is backward-compatible with WAL-E (WAL-G can restore WAL-E backups, but not vice versa). The Citus post states: "WAL-G is a complete rewrite that provides the same functionality as WAL-E, but boasts performance improvements of 4x faster restores on recovery." Non-exclusive mode (Postgres 9.6+) lets WAL-G obtain `backup_label` and `tablespace_map` from the stop-backup query result rather than requiring an on-disk `backup_label`, eliminating the WAL-E-era problem where a crash during an exclusive backup left a stale `backup_label` that blocked startup. WAL-G also emphasizes unit tests over WAL-E's slow, network-dependent integration tests. In production comparison, GitLab's runbooks report that the "gitlab-restore" project defaults to WAL-G "as it gives 3-4 times better restoration speed than WAL-E," quantified as an expected PGDATA fill speed of ~0.5 TiB/hour for WAL-E versus ~2 TiB/hour for WAL-G on an n1-standard-16 instance. [GitHub](#) [Citus Data](#)

2. Language, module structure, dependencies, build system

WAL-G is written in Go. The master `go.mod` sets a Go 1.25.x directive with `toolchain go1.25.9` (a direct read of the master file shows `go`

1.25.8); confirm against the exact commit you build, as the patch level moves).

Key dependencies (from master (go.mod)):

- PostgreSQL driver: (github.com/jackc/pgx/v5 v5.10.0), plus (github.com/jackc/pglogrepl) for the replication protocol.
- Cloud SDKs: (github.com/aws/aws-sdk-go v1.55.7) (S3), (cloud.google.com/go/storage) (GCS), (github.com/Azure/azure-sdk-for-go/sdk/storage/azblob) (Azure), (github.com/aliyun/alibabacloud-oss-go-sdk-v2) (Alicloud OSS), (github.com/ncw/swift/v2) (Swift), (github.com/pkg/sftp) (SSH), (github.com/yandex-cloud/go-sdk) (YC KMS).
- CLI/config: (github.com/spf13/cobra v1.7.0), (github.com/spf13/viper v1.7.0), (github.com/spf13/pflag), (github.com)
- Compression: (github.com/pierrec/lz4/v4), (github.com/ulikunitz/xz) (LZMA), (github.com/klauspost/compress) (zstd), (github.com/google/brotli/go/cbrotli) (cgo, build tag), (github.com/cyberdelia/lzo v1.0.0) (cgo, GPL, build tag).
- Crypto: (github.com/ProtonMail/go-crypto v1.3.0) (OpenPGP), (github.com/minio/sio v0.2.0) (AES DARE streaming), (golang.org/x/crypto), (github.com)
- Other DBs: (github.com/go-mysql-org/go-mysql), (github.com/microsoft/go-mssqldb), (go.mongodb.org/mongo-driver), (github.com/redis/go-redis/v9), (github.com/apache/cloudberry-go-libs), (github.com)
- Data structures: (github.com/RoaringBitmap/roaring/v2) (delta-map bitmaps), (github.com/hashicorp/golang-lru/v2).
- Logging/metrics: (github.com/wal-g/tracelog), (github.com/cactus/go-statsd-client/v5), (github.com/prometheus/client_golang), (github.com/pkg/profile).
- Direct I/O: (github.com/ncw/directio).

(Note: some transitively pinned cloud SDK patch versions vary between the (go.mod) (require) blocks and vendored copies; verify against your build tag if exact SDK versions matter.)

Makefile. (PKG := github.com/wal-g/wal-g), (BUILD_TAGS) is assembled conditionally: (ifdef USE_BROTLI → brotli), (ifdef USE_LIBSODIUM → libsodium), (ifdef USE_LZO → lzo). Each engine has a (MAIN_<DB>_PATH) (e.g. (main/pg)) where (go build -mod vendor -tags "\$(BUILD_TAGS)" -o wal-g -ldflags "-s -w -X .../cmd/pg.buildDate=... -X .../cmd/pg.gitRevision=... -X .../cmd/pg.walgVersion=...") runs. So version/date/revision are injected at link time into the per-engine (cmd/<db>) package variables ((walgVersion), (gitRevision), (buildDate)). (make deps), (make unittest), (make coverage), and integration targets ((pg_integration_test), etc.) round out the build. The compiled binary lives at (main/pg/wal-g); (make pg_install) honors (GOBIN).

3. Repository structure

- (main/<db>/) — thin (package main) entrypoints per engine; each calls the corresponding (cmd/<db>.Execute()).
- (cmd/<db>/) — Cobra command trees per engine. (cmd/pg/pg.go) defines the root (Cmd = &cobra.Command{Use: "wal-g", ...}) with a (PersistentPreRun) that calls (internal.AssertRequiredSettingsSet()) and sets WAL size from (PgWalSize). (cmd/pg) contains one file per command: (backup_push.go), (backup_fetch.go), (backup_list.go), (backup_mark.go), (catchup_fetch.go), (catchup_push.go), (catchup_list.go), (copy.go), (daemon.go), (delete.go), (wal_fetch.go), (wal_prefetch.go), (wal_push.go), (wal_receive.go), (wal_restore.go), (wal_show.go), (wal_verify.go), plus (pgbackrest*.go). (cmd/common) holds shared wiring. (GitHub)
- (internal/) — engine-agnostic core: the (Uploader), (TarBallQueue), (TarBall)/(TarBallMaker) interfaces, (CompressAndEncrypt)/(DecryptAndDecompressTar), (ExtractAll), delete handler, backup-mark handler, reader makers, sentinel/serializer helpers, and (StorageAdapters) (the table mapping (S3_PREFIX)/(FILE_PREFIX)/(GS_PREFIX)/(AZ_PREFIX)/(SWIFT_PREFIX)/(SSH_PREFIX) to (ConfigureFolder) functions).
 - (internal/config) — Viper-backed settings; defines all (WALG_*) constants (e.g. (DownloadConcurrencySetting = "WALG_DOWNLOAD_CONCURRENCY")), per-engine default maps ((PgDefaultSettings{PgWalSize: "16"}), (MongoDefaultSettings), (SQLServerDefaultSettings), (GPDefaultSettings)) and the (AllowedSettings)/(CommonAllowedSettings) allow-lists. (Go Packages) (Go Packages)
 - (internal/databases/postgres) — the Postgres-specific engine: (Bundle), tar-ball composers, (IncrementalPageReader), (PagedFileDeltaMap), (PgQueryRunner), (FileTarInterpreter), (BgUploader), delta file manager, WAL-verify runners, catchup unwrappers, and constants ((VersionStr="005"), (BaseBackupsPath="/basebackups_005/"), (WalPath="/wal_005/"), (SentinelSuffix="_backup_stop_sentinel.json"), (DatabasePageSize), (RelFileSizeBound=1<<30)). Subpackage (orioledb). (Go Packages) (Go Packages)
 - (internal/databases/{mysql,mssql,mongo,redis,greenplum,fdb,etc}) — per-engine logic.
 - (internal/walparser) — parses WAL segments: (walParser.ParseRecordsFromPage), (XLogPage)/(XLogPageHeader)/(XLogRecord), (RelFileNode), (BlockLocation), with (WalPageSize = BlockSize = 8192), (XLogRecordAlignment = 8). (Go Packages)
 - (internal/compression) — compressor/decompressor registry.
 - (internal/crypto) — the (Crypter) interface and its (openpgp), (libsodium), (aws kms), (yckms), and (envelope/*) implementations.

- `internal/multistorage` — failover storage folder wrappers and the aliveness cache.
- `pkg/storages/` — the storage abstraction (a Git subtree formerly the standalone `wal-g/storages` repo): `storage` (the `Folder`/`Object` interfaces), and backend packages `s3`, `gcs`, `azure`, `oss`, `swift`, `fs`, `sh` (SSH), `memory` (for tests).
- `utility/` — small shared helpers (path handling, `Empty` sets, etc.).
- Supporting: `submodules/` (e.g. brotli/libsodium C sources), `docker/`, `docs/`, `test/`, `tests_func/` (Gherkin/godog), `testtools/` (mocks), `benchmarks/`.

4. Commands

Common to all engines: `backup-list` (`--pretty`, `--json`, `--detail`), `delete` (dry-run by default, `--confirm` to execute), and `st` storage tools. [Readthedocs](#)

`delete` modes: `retain [FULL|FIND_FULL] N [--after name|time]`, `before [FIND_FULL] name`, `everything [FORCE]`, `target [FIND_FULL] name | --target-user-data DATA`, and (Postgres) `garbage [ARCHIVES|BACKUPS]`. Permanent backups are excluded unless `FORCE`. A global `--use-sentinel-time` flag orders backups by sentinel start time rather than the timeline+segment embedded in the backup name — important after `pg_upgrade` resets the timeline, where name-order can otherwise cause `retain` to delete newer backups. [GitHub](#)

PostgreSQL command set: `backup-push [PGDATA]` (full or delta; `--permanent`, `--full`, `--verify`, `--delta-from-name`, `--delta-from-user-data`, `--without-files-metadata`, `--add-user-data`); `backup-fetch PATH NAME [LATEST]` (`--reverse-unpack`, `--skip-redundant-tars`, `--target-user-data`, partial restore by database/table); `backup-mark -b NAME [-i]`; `wal-push PATH`; `wal-fetch NAME DEST`; `wal-prefetch` (internal, hidden — forked by `wal-fetch`); `wal-show`; `wal-verify [integrity] [timeline]`; `wal-receive` (RPO=0 replica-style receiver, beta); `wal-restore` (restores WAL segments needed for `pg_rewrite`); `catchup-push`/`catchup-fetch`/`catchup-send`/`catchup-receive`/`catchup-list`; `copy`; `daemon`; `st`; and `pgbackrest backup-list`/`backup-fetch`/`wal-fetch`/`wal-show` (read `pgBackRest` repos; full backups only).

Other engines: MySQL/MariaDB use `backup-push`/`xtrabackup-push` (delta only supported via `xtrabackup-push`), `backup-fetch`, `binlog-push`, `binlog-replay`, `binlog-server`, `binlog-list`. MongoDB/Redis/SQL Server/Greenplum have their own analogous stream-oriented command sets (e.g. Greenplum adds restore-point commands and AO/AOCS integrity checks). [Readthedocs](#)

5. Main workflow pathways

Full backup (`backup-push`)

Two streaming options: (1) run as the `postgres` OS user reading `PGDATA` directly from the filesystem — high performance and required for delta backups and partial restore; (2) stream through the Postgres `BASE_BACKUP` replication protocol (`--pgghost`/`PGHOST`, leave data dir out), which runs remotely as a separate user but is single-threaded and cannot do delta backups. WAL-G validates that the command argument, `PGDATA` env var, and config agree. [Readthedocs](#) [GitHub](#)

Sequence (`HandleBackupPush`) → (`Bundle`):

1. `PgQueryRunner.StartBackup` issues the non-exclusive start (`pg_backup_start`/`pg_start_backup` depending on version; `BuildStartBackup` formats the query per server version), returning the start LSN, whether the server is in recovery (replica), and the data dir. [Go Packages](#) [Go Packages](#)
2. `Bundle.StartQueue` creates a `TarBallQueue`; the bundle walks `PGDATA` (`HandleWalkedFSObject`), skipping excluded paths (`pg_dynshmem`, `pg_notify`, `pg_replslot`, `pg_serial`, `pg_stat_tmp`, `pg_snapshots`, `pg_subtrans`, `pgsql_tmp`, `postmaster.pid`, `postmaster.opts`, `recovery.conf`, `postgresql.auto.conf.tmp`, etc.). [GitHub](#)
3. Files are handed to a **tar-ball composer** which packs them into tar members `part_00i.tar.<ext>`. Each tarball except the last is at least `TarSizeThreshold` bytes (`WALG_TAR_SIZE_THRESHOLD`, default 1 GiB = `(1<<30 - 1)`). [Go Packages](#)
4. `pg_control` is packed **last** (it is the sentinel guaranteeing a complete backup), then `PgQueryRunner.StopBackup` returns the `backup_label`, `tablespace_map`, and stop LSN, which are written as final tar members. [Go Packages](#)
5. WAL-G writes the JSON sentinel `<name>_backup_stop_sentinel.json` and extended metadata. If the backup ran on a standby, `Bundle.CheckTimelineChanged` compares timelines at start/stop; on a promotion or timeline switch WAL-G uploads data but refuses to finalize the sentinel and exits with an error. [GitHub](#)

Tar-ball composer strategies (chosen via env):

- **Regular** (`RegularTarBallComposer`) — default; packs files in walk order.
- **Rating** (`WALG_USE_RATING_COMPOSER`, `RatingTarBallComposer`) — groups files with similar update frequency into the same tarballs (via a `ComposeRatingEvaluator` and per-relation `RelFileStatistics`), so that during `backup-fetch` whole redundant tarballs can be skipped. Slower to create. [Readthedocs](#)

- **Copy** (`WALG_USE_COPY_COMPOSER`, `CopyTarBallComposer`) — for full backups, server-side/remote-copies unchanged tarballs from the previous full backup (GSOC 2021 remote-copy work); falls back to regular if no prior full backup. Most effective for large cold datasets; effectiveness improves with smaller `WALG_TAR_SIZE_THRESHOLD`. [Readthedocs + 2](#)
- **Database** (`WALG_USE_DATABASE_COMPOSER`) — separates files from different directories in the default tablespace into different tars to speed partial restore. [Readthedocs](#)

`WALG_WITHOUT_FILES_METADATA` (or `--without-files-metadata`) disables per-file metadata tracking, saving GBs of RAM on clusters with >100k files, at the cost of being incompatible with delta backups. [Readthedocs](#)

Delta / incremental backups

Enabled by `WALG_DELTA_MAX_STEPS > 0` (max deltas between fulls, default 0). `WALG_DELTA_ORIGIN = LATEST` (chain each delta onto the previous, default) or `LATEST_FULL` (each delta is relative to the base full). Base selection can be overridden with `--delta-from-name` (`WALG_DELTA_FROM_NAME`) or `--delta-from-user-data` (`WALG_DELTA_FROM_USER_DATA`). [Readthedocs](#) [Readthedocs](#)

Delta detection is **page-level**. `IncrementalPageReader` (fields `PagedFile`, `FileSize`, `Lsn`, `Next`, `Blocks []uint32`) reads each relation file's 8 KiB pages; `SelectNewValidPage` validates the page header and, if the page LSN is $\geq$ the increment base LSN, appends the block number to the `Blocks` list. The reader "constructs a difference map during initialization and then re-reads the file"; the diff map may be up to `1GB/PostgresBlockSize` = 512 KiB of block entries per 1 GiB segment. `WriteDiffMapToHeader` prepends the changed-block map, and only changed pages are written to the increment file (magic byte `SignatureMagicNumber = 0x55`). Delta computation combines filesystem `ModTime` and page LSNs. (Known edge: if there is zero write activity since the base, the delta start LSN can equal/precede the base finish LSN and `backup-push` fails with "Finish LSN ... greater than current LSN".) [Go Packages](#) [Readthedocs](#)

WAL-based delta map. Instead of scanning every page, WAL-G can build the changed-block set by scanning WAL. `WALG_USE_WAL_DELTA` enables the `walDeltaRecordingReader`, which wraps the WAL stream during `wal-push`, parses records with `walparser`, and records `BlockLocation`s into `_delta/_part` files via the `DeltaFileManager`. On delta backup, `PagedFileDeltaMap` (a `map[walparser.RelFileNode]*roaring.Bitmap`) is built by `GetDeltaMap`/`GetLocationsFromWals`/`GetLocationsFromDeltas`, and `GetDeltaBitmapFor(filePath)` yields the roaring bitmap of changed blocks per relation, letting WAL-G read only those blocks. `SelectRelFileBlocks` filters the bitmap per relfilenode segment. [Go Packages](#) [Go Packages](#)

On restore, deltas are transparent: `backup-fetch` automatically pulls the base and all intermediate deltas and reconstructs a valid cluster (you still need WAL from the last backup's start to reach consistency). [Readthedocs](#)

WAL archiving (`wal-push`)

Wired as `Postgres archive_command = 'wal-g wal-push %p'`. Beyond uploading the requested segment, WAL-G's `BgUploader` opportunistically uploads adjacent already-completed segments it finds next to the requested file (looking at `.ready` files in `archive_status`). `NewBgUploader(walFilePath, maxParallelWorkers, maxNumUploaded, uploader, preventWalOverwrite, readyRename)` bounds concurrency (`maxParallelWorkers`) and total work per invocation (`maxNumUploaded`); `TOTAL_BG_UPLOADED_LIMIT` overrides the default cap (32 files per scan). `WALG_PREVENT_WAL_OVERWRITE` makes WAL-G check whether a differently-named-but-existing segment is already archived and return non-zero to stop Postgres deleting local WAL. `WALG_UPLOAD_WAL_METADATA` controls WAL metadata emission (`BULK`/`INDIVIDUAL`/`NOMETADATA`). [Readthedocs](#) [Readthedocs](#)

WAL restore (`wal-fetch`)

Wired as `restore_command = 'wal-g wal-fetch %f %p'`. WAL-G prefetches segments ahead of the requested one into `./wal-g/prefetch` (or `pg_wal` by default), forking hidden `wal-prefetch` processes; a `wal-fetch` both serves from and refills the cache, and evicts segments older than the last requested one to bound cache size. Caveat: with `pg_rewind` on Postgres 13 (which itself invokes `restore_command`), prefetched WAL can cause false failures — mitigate with `WALG_DOWNLOAD_CONCURRENCY=1` or a prefetch folder outside `PGDATA`. [Readthedocs](#)

Restore (`backup-fetch`)

`wal-g backup-fetch PATH NAME|LATEST` (creates the target dir tree). `FileTarInterpreter.Interpret` extracts each tar member; `ExtractAll` handles `.lz4`/`.lzma`/`.lzo`/`.zst`/`.br`/`.tar` (`.nop` for tests), extracting each file in its own goroutine and retrying failures $\sim \log_2(\text{concurrency})$ times, halving concurrency each retry. `WALG_DOWNLOAD_FILE_RETRIES` (default 15) bounds per-file retries. [GitHub](#)

Delta application on fetch: increments are applied over the base. Two file unwrappers exist — `DefaultFileUnwrapper` (`backup-fetch`) and `CatchupFileUnwrapper` (`catchup`). `WritePagesFromIncrement`/`RestoreMissingPages` write changed pages into the reconstructed relation file. **Reverse delta unpack** (`WALG_USE_REVERSE_UNPACK`/`--reverse-unpack`, beta since v0.2.19) unpacks from the newest delta back to the base, so each block is written once (copy-on-first-write), avoiding rewriting blocks superseded by later deltas. Combined with **redundant-tar skipping** (`WALG_SKIP_REDUNDANT_TARS`/`--skip-redundant-tars`, best paired with the rating composer), whole tarballs whose every file is superseded can be skipped. [GitHub](#)

Partial restore: pass `(database)` or `(database/namespace.table)` targets; WAL-G restores only those relations' files (`(--skip-redundant-tars)` and `(--reverse-unpack)` are enabled automatically). Because catalog rows for unrestored objects remain, dropping them afterward is recommended. `(--target-user-data)` `(WALG_FETCH_TARGET_USER_DATA)` selects a backup by sentinel UserData.

Storage layer

`(pkg/storages/storage)` defines the `(Folder)` and `(Object)` abstractions; `(internal.StorageAdapters)` maps a prefix setting to a backend `(ConfigureFolder)`. Backends: **S3** `(WALG_S3_PREFIX)` `(WALE_S3_PREFIX)`; AWS SDK credential chain; options for region auto-detect, path-style, storage class, SSE/SSE-KMS, CSE-KMS, object-lock retention, ListObjects v1/v2, range-GET resumable downloads via `(WALG_S3_RANGE_BATCH_ENABLED)`, `(S3_DELETE_BATCH_SIZE)`), **GCS** `(WALG_GS_PREFIX)`; `(GCS_MAX_CHUNK_SIZE)` default 50 MiB, CSEK, `(GCS_NORMILIZE_PREFIX)` for WAL-E compatibility), **Azure Blob** `(WALG_AZ_PREFIX)`+`(AZURE_STORAGE_ACCOUNT)`; buffer size/count tuning, cloud-environment selection), **Alicloud OSS** `(WALG_OSS_PREFIX)`; multipart), **Swift** `(WALG_SWIFT_PREFIX)`), **filesystem** `(WALG_FILE_PREFIX)`), and **SSH/SFTP** `(WALG_SSH_PREFIX)`+`(SSH_PORT)` `(SSH_USERNAME)` `(SSH_PASSWORD)` `(SSH_PRIVATE_KEY_PATH)`. [Go Packages](#)

Multi-storage / failover `(internal/multistorage)`: a nested `(WALG_FAILOVER_STORAGES)` config section (config-file only) defines named secondary storages; the primary (config root) is used when alive, otherwise the first alive failover storage in lexicographic name order. WAL-G maintains a storage-aliveness cache in memory and in a file shared between runs, updated by explicit RW checks `(WALG_FAILOVER_STORAGES_CHECK_TIMEOUT)` 30s, `(WALG_FAILOVER_STORAGES_CHECK_SIZE)` 1 MiB) and by an EMA of live operation outcomes `(new_aliveness = reported*α + prev*(1-α))`, with tunable alive/dead limits and α bounds `(WALG_FAILOVER_STORAGES_CACHE_*)`, cache TTL `(WALG_FAILOVER_STORAGES_CACHE_LIFETIME)` 15m). `(WALG_FAILOVER_STORAGES_CHECK=false)` treats all storages as alive. Failover is supported by `(wal-push)`, `(wal-fetch)`, `(wal-prefetch)`, `(backup-push)`, `(backup-fetch)`, `(backup-list)`, and `(delete)`. `(wal-g st transfer)` moves files between storages (e.g. `(transfer backups)`, `(transfer pg-wals)`) to reconcile a failover storage back to primary, moving the `(_backup_stop_sentinel.json)` last and deleting it from target first for consistency. [Readthedocs](#)

Compression

`(WALG_COMPRESSION_METHOD)` ∈ `(lz4)` (default, fastest, weak ratio), `(lzma)` (~6× better than lz4, much slower), `(zstd)`, `(brotli)` (both ~3× better than lz4, balanced), or `(none)`. Compression is pluggable through the `(compression.Compressor)` `(Decompressor)` interfaces registered by algorithm name and file extension `(Lz4FileExtension="lz4")`, `(LzmaFileExtension="lzma")`, `(ZstdFileExtension="zst")`, `(LzoFileExtension="lzo")`. LZO is decompress-only and GPL-3.0 (build tag `(lzo)`); brotli requires the `(brotli)` cgo build tag. [GitHub](#) [Go Packages](#)

Encryption

Encryption is selected by `(internal.ConfigureCrypter)` (→ `(ConfigureCrypterForSpecificConfig)`) from env/config, first match wins, and is completely orthogonal to compression. The `(crypto.Crypter)` interface is:

```
go

type Crypter interface {
    Name() string
    Encrypt(writer io.Writer) (io.WriteCloser, error)
    Decrypt(reader io.Reader) (io.Reader, error)
}
```

Implementations and precedence:

1. **OpenPGP** `(internal/crypto/openpgp)` — `(WALG_PGP_KEY)` `(WALG_PGP_KEY_PATH)` (+`(WALG_PGP_KEY_PASSPHRASE)`), or the deprecated key-ring `(GPG_KEY_ID)` `(WALG_GPG_KEY_ID)` path which shells out to `(gpg)` `(GpgBin)` and caches the public keyring in `(~/.walg_key_cache)`. On master this uses github.com/ProtonMail/go-crypto/openpgp (a drop-in fork of golang.org/x/crypto/openpgp). Public key for push, private (which also contains public) for fetch. [GitHub](#) + 2
2. **Envelope PGP** `(internal/crypto/envelope/openpgp)`, with enveloper subpackages `(.../enveloper/yckms)` and `(.../enveloper/cached)` — `(WALG_ENVELOPE_PGP_KEY)` `(_PATH)`: the armored PGP key itself is stored KMS-encrypted+base64 and injected into backup/WAL metadata; requires a Yandex Cloud KMS key `(WALG_ENVELOPE_PGP_YC_CSE_KMS_KEY_ID)`, with a TTL cache `(WALG_ENVELOPE_CACHE_EXPIRATION)`, default 0 = permanent in memory). Plain PGP and envelope PGP are mutually exclusive (hard error). [Readthedocs](#) [GitHub](#)
3. **AWS KMS envelope** `(internal/crypto/awskms)` — `(WALG_CSE_KMS_ID)` (+`(WALG_CSE_KMS_REGION)`): generates a local data key (implements the `(crypto.SymmetricKey)` interface with `(Generate)` `(Encrypt)` `(Decrypt)` `(GetEncryptedKey)` `(SetEncryptedKey)`), has KMS encrypt it, stream-encrypts data (AES DARE via github.com/minio/sio), and stores the encrypted data key alongside for KMS-decrypt on fetch.

4. **Yandex KMS** (`(internal/crypto/yc kms)`) — `(YC_CSE_KMS_KEY_ID)(+(YC_SERVICE_ACCOUNT_KEY_FILE))`; also envelope-style via `(github.com/yandex-cloud/go-sdk)`.
5. **libsodium** (`(internal/crypto/libsodium)`, build tag `(libsodium)`) — `(WALG_LIBSODIUM_KEY)(_PATH)` with `(WALG_LIBSODIUM_KEY_TRANSFORM) ∈ (base64)(hex)(none)`; uses `(crypto_secretstream_xchacha20poly1305)` (`(init_push/init_pull)`) with a fixed 32-byte key (no KDF). Server-side S3 SSE (`(WALG_S3_SSE)`, `(WALG_S3_SSE_KMS_ID)`) is orthogonal to client-side encryption. `(GitHub)`

Retention & deletion

See §4 for modes. `(delete before FIND_FULL)` computes the minimum full needed to keep all deltas alive; without `(FIND_FULL)` a call that would orphan deltas fails listing them. `(target)` deletes a named/user-data backup and its dependent deltas (or, with `(FIND_FULL)`, all backups sharing its base) and, unlike other modes, does not delete WALs. `(delete garbage [ARCHIVES|BACKUPS])` cleans up outdated WALs and leftover/partial backup files before the earliest non-permanent backup — useful after `(target)` deletes. Permanent backups (set at push via `(--permanent)` or later via `(backup-mark -b NAME, -i)` to unmark) are preserved unless `(everything FORCE)`; marking a delta permanent marks all its parents permanent (`(GetBackupsToMark)(MarkBackup)`). `(GitHub + 2)`

Verification

`(backup-push --verify)(WALG_VERIFY_PAGE_CHECKSUMS)` validates each data page's checksum during backup (FNV-1a based, `(NSums=32)` parallel sums, `(FNVPrime=16777619)`, checksum at page offset `(PdChecksumOffset=8)`, `(PdChecksumLen=2)`); up to 10 corrupt block numbers per file are recorded in the sentinel JSON (`(WALG_STORE_ALL_CORRUPT_BLOCKS)` to store all). `(wal-verify integrity)` connects to the cluster, gets the current segment+timeline, walks WAL history in reverse, and classifies gaps as `(FOUND)`, `(MISSING_DELAYED)` (within `(WALG_INTEGRITY_MAX_DELAYED_WALS)` of current LSN — Postgres likely hasn't archived yet), `(MISSING_UPLOADING)` (within `(WALG_UPLOAD_CONCURRENCY)` of the delayed boundary — likely still uploading), or `(MISSING_LOST)` (unrecoverable). Overall status: `(OK)(WARNING)(FAILURE)`. `(wal-verify timeline)` checks the current timeline ≥ storage timelines (split-brain detection). `(wal-show)` lists segments per timeline with optional backup listing. `(Go Packages + 4)`

Daemon mode & socket communication

`(wal-g daemon SOCKET)` (Postgres) runs a long-lived process that services archive/fetch requests over a Unix socket, working with the PostgreSQL 15+ archive library `(walg_archive)` or the `(walg-daemon-client)` binary — improving parallel archiving efficiency by avoiding process spawn per segment. It uses `(systemd(SdNotify))` (sent in a goroutine) for readiness. `(WALG_ARCHIVE_TIMEOUT)` (default 60s) bounds each in-daemon WAL operation.

Catchup (standby resync)

`(catchup-push /master/PGDATA --from-lsn REPLICA_LSN)` builds an incremental (delta-like) backup on the primary containing only blocks changed since the stopped replica's LSN (obtained e.g. via `(pg_controldata)`); `(catchup-fetch /replica/PGDATA BACKUP_NAME)` applies it to the stopped replica. There is also a live streaming pair: `(catchup-receive $PGDATA_STANDBY PORT)` on the standby and `(catchup-send $PGDATA_PRIMARY host:PORT)` on the primary. `(catchup-list)` lists catchup backups. `(Readthedocs + 2)`

6. Concurrency model

- **Upload:** `(WALG_UPLOAD_CONCURRENCY)` (default 16) concurrent upload streams; `(WALG_UPLOAD_DISK_CONCURRENCY)` (default 1) disk-read streams; `(WALG_UPLOAD_QUEUE)` bounds prepared tarballs awaiting upload. The `(TarBallQueue)` (`(tarsToFillQueue)(uploadQueue)` channels, `(parallelTarballs)`, `(maxUploadQueue)`, guarded by `(sync.Mutex)` + `(atomic.Bool started)` coordinates producers filling tarballs and consumers uploading them. `(GitHub + 2)`
- **Download/extract:** `(WALG_DOWNLOAD_CONCURRENCY)` (default `(min(#files, 10))`); each file extracted in its own goroutine with retry/back-off. `(GitHub)` `(Readthedocs)`
- **WAL background upload:** `(BgUploader)` worker pool bounded by `(maxParallelWorkers)` and `(maxNumUploaded)(TOTAL_BG_UPLOADED_LIMIT)`.
- **Rate limiting:** `(WALG_NETWORK_RATE_LIMIT)` (bytes/s, applied to backup-push/fetch via `(golang.org/x/time/rate)`) and `(WALG_DISK_RATE_LIMIT)`.
- **Profiling:** `(PROFILE_MODE)` (cpu/mem/mutex/block/goroutine/...), `(PROFILE_SAMPLING_RATIO)`, `(PROFILE_PATH)`.

7. Configuration system

Two mechanisms via Viper: environment variables or a config file (`(--config PATH)` or `(WALG_CONFIG_PATH)`; JSON/YAML/envfile/etc.). Every documented variable can be set either way; failover storages must use a config file (nested structure). Postgres connection uses standard `(PGHOST)(PGPORT)(PGUSER)(PGPASSWORD)(PGPASSFILE)(-/ .pgpass)` (Unix socket if `(PGHOST=/var/run/postgresql)`). `(AssertRequiredSettingsSet)` enforces that a storage prefix is configured. Notable settings appear throughout above; monitoring via

(WALG_STATSD_ADDRESS)/(WALG_STATSD_EXTRA_TAGS); troubleshooting via (WALG_LOG_LEVEL=DEVEL) and (S3_LOG_LEVEL=DEVEL). Both (WALG_) and legacy (WALE_) forms are accepted for several variables ((WALE_S3_PREFIX), (WALE_GPG_KEY_ID)). (GitHub + 2)

8. Notable internals

- **Page-file parsing / LSN handling:** (DatabasePageSize = walparser.BlockSize = 8192), (RelFileSizeBound = 1<<30) (1 GiB segment), (BlocksInRelFile = RelFileSizeBound/DatabasePageSize). (IncrementalPageReader) reads the page header LSN to decide inclusion. (walparser) reconstructs (XLogRecord)s across pages ((ParseRecordsFromPage), handling (PartialPageError)/(ZeroPageError), WAL-switch records) and yields (BlockLocation{RelFileNode, BlockNo}) for delta maps. (DefaultSpcNode = 1663) (default tablespace OID). The (xid64) page-layout patch is supported.
- **Tar interpreter:** (FileTarInterpreter) reconstructs files, distinguishing new vs. incremental members and applying page increments.
- **Sentinel files:** (<name>_backup_stop_sentinel.json) (constant (SentinelSuffix)) marks a complete backup. The sentinel DTO carries (LSN), (FinishLSN), (PgVersion), (UserData), (Files) (the (BackupFileList)), and for deltas (DeltaFromLSN)/(DeltaFrom)/(DeltaFullName)/(DeltaCount) (historically (S3TarBallSentinelDto); (IsIncremental()) tests delta-ness). (pg_control) is the internal completeness sentinel packed last.
- **Storage layout:** (VersionStr = "005"); base backups under (basebackups_005/) (each (base_<segment>) or delta (base_<seg>_D_<baseSeg>) plus its (<name>_backup_stop_sentinel.json) and (tar_partitions/), WAL under (wal_005/). (WALG_STORAGE_PREFIX) adds a path prefix within the bucket.
- **Copy-on-write / incremental merge on fetch:** reverse-delta unpack writes each block exactly once from newest→oldest, and redundant-tar skipping avoids downloading fully-superseded tarballs — the fetch-side complement to the rating composer.

Recommendations

Baseline production Postgres setup. Run (wal-g wal-push) from (archive_command) and take (backup-push) on a replica (offloads I/O from the primary). Use (WALG_COMPRESSION_METHOD=zstd) for the best speed/ratio balance (fall back to (lz4) if CPU-bound on WAL archiving, or (lzma) (brotli) if storage cost dominates and backup windows are generous). Set (WALG_UPLOAD_CONCURRENCY) to match available cores/network and keep (WALG_UPLOAD_DISK_CONCURRENCY) low on spinning disks.

Delta strategy. For large, mostly-cold clusters enable (WALG_DELTA_MAX_STEPS) (e.g. 6–7 daily deltas between weekly fulls) with (WALG_DELTA_ORIGIN=LATEST), and turn on (WALG_USE_REVERSE_UNPACK) + (WALG_USE_RATING_COMPOSER) + (WALG_SKIP_REDUNDANT_TARS) to cut restore I/O. For very high-write clusters, prefer the copy composer or shorter delta chains. Consider (WALG_USE_WAL_DELTA) to build delta maps from WAL rather than full page scans if backup-push disk scanning is the bottleneck (validate carefully — it depends on complete archived WAL). If you have >100k files and don't need deltas, (WALG_WITHOUT_FILES_METADATA) sharply cuts RAM.

Encryption. Prefer client-side encryption: libsodium (build with (USE_LIBSODIUM) and supply a real 32-byte key via (WALG_LIBSODIUM_KEY_TRANSFORM=base64)) or OpenPGP; for regulated environments use the AWS/YC KMS envelope crypters so the data key is KMS-wrapped. Verify your release binary actually includes libsodium — stock GitHub binaries historically did not, and a configured key with no support compiled in now fails hard.

Verification & safety. Schedule (wal-verify integrity timeline) (e.g. hourly) and alert on (FAILURE); run (backup-push --verify) periodically to catch on-disk page corruption that Postgres only notices on read. Automate periodic full restore tests. Use (delete retain FULL N) (not bare (retain N), which fails on deltas), and after any (pg_upgrade)/timeline reset add (--use-sentinel-time) so retention doesn't delete newer backups. Mark disaster-recovery fulls (--permanent).

Thresholds that change the plan. If daily (backup-push) approaches your backup window — GitLab's runbooks state "As of September 2021, the daily backup process performed by WAL-G takes ~9 hours with Postgres cluster size ~14 TiB" — move to deltas + copy/rating composer or a dedicated backup replica. If WAL archiving lags (rising (MISSING_UPLOADING) in (wal-verify)), raise (WALG_UPLOAD_CONCURRENCY)/(TOTAL_BG_UPLOADED_LIMIT) or adopt daemon mode with (walg_archive) on PG15+. If object-storage availability is a risk, configure (WALG_FAILOVER_STORAGES) and reconcile with (st transfer) when the primary recovers.

Caveats

- **Version drift:** (go.mod) on master shows a Go 1.25.x directive ((go 1.25.8) in a direct read of the file) with (toolchain go1.25.9) and modern dependency versions as of mid-2026; older deployed builds (v2.x, v1.x) differ (e.g. default (WALG_UPLOAD_CONCURRENCY) was 10 in older docs, 16 now; earlier releases lacked failover storage, daemon mode, envelope PGP). One enrichment pass reported slightly different transitive cloud-SDK patch versions (e.g. (cloud.google.com/go/storage v1.10.0), (azblob v1.6.1)) than the direct file read ((storage v1.63.0), (azblob v1.8.0)); confirm against the exact commit you deploy. (Go Packages)
- Some struct signatures (e.g. (IncrementalPageReader), (Bundle), (S3TarBallSentinelDto)) are quoted from pkg.go.dev indexes of tagged releases and community forks; field sets have evolved across versions (the sentinel DTO type/fields and (Bundle) fields changed over

time). Treat them as representative, not byte-exact for master HEAD. The `crypto.Crypter` interface and base crypters are confirmed unchanged from the last indexed tag through master; envelope-crypter file layout is inferred from `configure.go` imports rather than a fetched source file.

- The BASE_BACKUP protocol streaming mode is single-threaded and cannot do delta backups or (fully) partial restore — the filesystem mode is required for those.
- Reverse-delta unpack, redundant-tar skipping, `wal-receive`, WAL-based deltas, MySQL delta backups, and MongoDB/Redis/etcd support are variously marked beta/WIP in the docs; validate before production reliance.
- The Citus "4× faster than WAL-E" figure and the GitLab "~9h/14 TiB" and "3-4× better restoration speed / 0.5 vs 2 TiB/hour" figures are vendor/operator claims from specific historical contexts, not guarantees.
- The subagent noted a likely benign variable-naming swap in `ConfigureCrypterForSpecificConfig` (envelope key vs. key-path locals) that does not change behavior.